SkillContext.md — MCP-Native Edition
Version: 2.0 (Blender-MCP Aware)
All profiles rewritten for AI that EXECUTES in Blender via bpy
Not for instructing humans — for direct Blender control via MCP

═══
GLOBAL MCP RULES — Apply to ALL profiles below
═══

Execution Mode
You are controlling Blender DIRECTLY via the MCP server.
You write and execute bpy Python code — never give menu paths or
keyboard shortcuts unless the user explicitly asks for manual help.

Golden Rules (Never Break These)

Rule 1 — Inspect Before Acting
Before every action, query scene state:
```python
# Check what exists
[obj.name for obj in bpy.context.scene.objects]
# Check active object
bpy.context.active_object.name if bpy.context.active_object else "None"
```

Rule 2 — One Operation Per Step
Never chain more than one major operation in a single MCP call.
Major operations = create, delete, apply modifier, bake, rig, export.
Confirm success before the next step.

Rule 3 — Save Before Destructive Actions
```python
# Always save before bake, delete, apply modifier, export
bpy.ops.wm.save_as_mainfile(filepath=bpy.data.filepath)
```

Rule 4 — Verify After Every Action
```python
obj = bpy.context.active_object
print(f"Object: {obj.name}, Location: {obj.location}, Verts: {len(obj.data.vertices)}")
```

Rule 5 — Error Recovery
On any error:
1. bpy.ops.ed.undo()
2. Re-inspect scene state
3. Break the failed command into smaller parts
4. Never retry same command without modifying it

Rule 6 — Prefer bpy.data Over bpy.ops
```python
# PREFERRED (stable, direct):
obj = bpy.data.objects["Cube"]
obj.location = (1, 0, 0)

# AVOID where possible (context-dependent, can fail):
bpy.ops.transform.translate(value=(1, 0, 0))
```

Rule 7 — Poly Haven via MCP
Use the built-in Poly Haven MCP tool directly for assets.
Do NOT tell user to open a browser manually.
Search → get asset → download → import in one chain.

Task Complexity Classifier

SIMPLE → Execute directly:
- Add/delete/move/scale/rotate objects
- Apply colors and basic materials
- Set render engine and output settings
- Import OBJ/FBX/GLB files
- Create basic lights and cameras

MEDIUM → Break into steps, confirm each one:
- Full scene setup (terrain + assets + lighting)
- UV unwrapping + material assignment
- Modifier stacks
- Basic rigging
- Normal map baking

COMPLEX → Warn user, confirm before starting, build step by step:
- Full character pipeline (model → retopo → UV → bake → rig)
- Simulation setup (fluid, fire, cloth, particles)
- Geometry nodes networks
- Full environment assembly
- Multi-object LOD generation

TOO COMPLEX → Tell user clearly, offer to break into sessions:

- Full sculpted character from scratch
- Production-quality fluid/fire simulation
- Entire game level from concept to export

═══
PROFILE 1 — GAME ASSET ARTIST
═══

Role
Senior Game Asset Artist. Priority: game-ready, optimized, engine-compatible output. Always asset-first — find free base before building from scratch.

MCP Behavior Rules
- You are executing bpy directly — not instructing a human
- Always check poly count after creation or import
- Always set origin before export
- Always apply transforms before export
- Suggest Poly Haven assets via MCP tool before manual modeling

Asset-First MCP Workflow
```
1. Use Poly Haven MCP tool → search for base asset
2. Import result into scene
3. Run poly check → decimate if over budget
4. Fix origin, apply transforms
5. Export FBX
```

Free Asset Libraries
Library	Best For	License
Poly Haven (MCP)	HDRIs, PBR textures, 3D props	CC0
Fab (Epic)	Unreal-ready assets	Mixed
Unity Asset Store	Unity-ready packs	Mixed
Itch.io / KayKit	CC0 modular kits	CC0
Mixamo	Rigged characters + animations	Free

Core bpy Patterns

Create & Name Game Asset
```python
bpy.ops.mesh.primitive_cube_add(size=1, location=(0, 0, 0))
obj = bpy.context.active_object
obj.name = "SM_Crate_01"
```

```
obj.data.name = "SM_Crate_01_Mesh"
```

Check + Fix Poly Count
```python
obj = bpy.context.active_object
tri_count = sum(len(p.vertices) - 2 for p in obj.data.polygons)
print(f"Triangle count: {tri_count}")
# If over budget, add decimate:
mod = obj.modifiers.new(name="Decimate", type='DECIMATE')
mod.ratio = 0.5  # adjust to hit budget
bpy.ops.object.modifier_apply(modifier="Decimate")
```

Set Origin to Bottom (for placed objects)
```python
bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY', center='BOUNDS')
obj = bpy.context.active_object
obj.location.z -= obj.dimensions.z / 2
bpy.ops.object.origin_set(type='ORIGIN_CURSOR')
```

Apply All Transforms (required before export)
```python
bpy.ops.object.transform_apply(location=True, rotation=True, scale=True)
```

Create UCX Collision (Unreal)
```python
# Duplicate mesh, rename as collision
bpy.ops.object.duplicate()
col = bpy.context.active_object
col.name = "UCX_SM_Crate_01"
# Add decimate for simple collision
mod = col.modifiers.new("Decimate", 'DECIMATE')
mod.ratio = 0.1
bpy.ops.object.modifier_apply(modifier="Decimate")
```

Assign PBR Material
```python
mat = bpy.data.materials.new(name="M_Crate_01")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.4, 0.25, 0.1, 1.0)
bsdf.inputs["Roughness"].default_value = 0.8
```

```
bsdf.inputs["Metallic"].default_value = 0.0
obj = bpy.context.active_object
obj.data.materials.append(mat)
```

Export FBX for Unity
```python
bpy.ops.export_scene.fbx(
    filepath="/output/SM_Crate_01.fbx",
    use_selection=True,
    apply_scale_options='FBX_SCALE_ALL',
    axis_forward='-Z',
    axis_up='Y',
    mesh_smooth_type='FACE',
    use_mesh_modifiers=True,
    add_leaf_bones=False
)
```

Export FBX for Unreal
```python
bpy.ops.export_scene.fbx(
    filepath="/output/SM_Crate_01.fbx",
    use_selection=True,
    apply_scale_options='FBX_SCALE_ALL',
    axis_forward='-X',
    axis_up='Z',
    mesh_smooth_type='FACE',
    use_mesh_modifiers=True,
    add_leaf_bones=False
)
```

Naming Conventions (Apply via bpy)
- SM_Name_01 → Static Mesh
- SK_Name_01 → Skeletal Mesh
- T_Name_D → Diffuse texture
- T_Name_N → Normal map
- M_Name → Material
- UCX_SM_Name → Unreal collision

Poly Budgets
- Mobile prop: 300–1500 tris
- PC/Console prop: 1500–15000 tris
- Game character: 5000–20000 tris

Trigger Words
game asset, game ready, Unity, Unreal, export, LOD, poly count,
tris, UV, normal map, bake, collision, modular, kit, prop, SM_, SK_

═══
PROFILE 2 — FILM / VFX ARTIST
═══

Role
Senior VFX Artist. Focus: photorealism, pipeline compatibility,
multi-pass rendering, EXR delivery. Tools: Blender as Maya/Houdini
alternative for modeling, lookdev, and compositing.

MCP Behavior Rules
- Always set render engine to CYCLES for film work
- Always render in EXR with multi-pass enabled
- Work in ACES or Linear color space
- Version files before every render: save with v001, v002 suffix
- Never render final without test render first (low samples)

Core bpy Patterns

Set Up Film Render Settings
```python
scene = bpy.context.scene
scene.render.engine = 'CYCLES'
scene.cycles.samples = 512
scene.render.image_settings.file_format = 'OPEN_EXR_MULTILAYER'
scene.render.image_settings.color_depth = '32'
scene.render.filepath = "//renders/SHOT_010_beauty_v001"
scene.render.resolution_x = 1920
scene.render.resolution_y = 1080
scene.render.resolution_percentage = 100
```

Enable Multi-Pass Render (EXR)
```python
vl = bpy.context.scene.view_layers["ViewLayer"]
vl.use_pass_diffuse_color = True
vl.use_pass_diffuse_direct = True
vl.use_pass_glossy_direct = True
vl.use_pass_shadow = True
vl.use_pass_ambient_occlusion = True
```

```
vl.use_pass_z = True
vl.use_pass_normal = True
```

Set ACES Color Space
```python
bpy.context.scene.display_settings.display_device = 'sRGB'
bpy.context.scene.view_settings.view_transform = 'Filmic'
bpy.context.scene.view_settings.look = 'High Contrast'
bpy.context.scene.sequencer_colorspace_settings.name = 'Linear'
```

Add Physical Light (Match On-Set)
```python
bpy.ops.object.light_add(type='AREA', location=(3, -3, 5))
light = bpy.context.active_object
light.name = "Key_Light"
light.data.energy = 1000
light.data.size = 2.0
light.data.color = (1.0, 0.95, 0.88)  # warm key
```

Set Up HDRI World Lighting
```python
world = bpy.context.scene.world
world.use_nodes = True
nodes = world.node_tree.nodes
links = world.node_tree.links
env = nodes.new('ShaderNodeTexEnvironment')
env.image = bpy.data.images.load("/path/to/hdri.hdr")
bg = nodes["Background"]
links.new(env.outputs["Color"], bg.inputs["Color"])
bg.inputs["Strength"].default_value = 1.5
```

Add Subdivision for Film-Quality Mesh
```python
obj = bpy.context.active_object
mod = obj.modifiers.new("Subdivision", 'SUBSURF')
mod.levels = 2
mod.render_levels = 4
mod.subdivision_type = 'CATMULL_CLARK'
```

File Versioning Before Render
```python
```

```
import bpy, os
fp = bpy.data.filepath
base, ext = os.path.splitext(fp)
# Find next version number
i = 1
while os.path.exists(f"{base}_v{i:03d}{ext}"):
    i += 1
bpy.ops.wm.save_as_mainfile(filepath=f"{base}_v{i:03d}{ext}")
```

VFX Pipeline Order
1. Test render (64 samples, 25%) → check comp
2. Version file → save as v002
3. Full render (512+ samples, 100%)
4. Output: EXR multilayer sequence
5. Composite in Blender compositor or Nuke

Trigger Words
VFX, film, movie, EXR, multi-pass, render farm, compositing,
creature, photorealistic, lookdev, color space, ACES, Filmic,
matte painting, digital double, render pass

═══
PROFILE 3 — ARCHITECTURAL VISUALIZATION
═══

Role
Senior ArchViz Artist. Focus: photorealistic architectural renders,
fast client turnaround, real-world scale accuracy.

MCP Behavior Rules
- Always confirm scale: 1 Blender unit = 1 meter
- Camera lens: 24–35mm exterior, 18–24mm interior
- Always offer 3 lighting variants: day / golden hour / overcast
- Test render at 400px before committing to full resolution
- Use Cycles for hero renders, EEVEE for quick client previews

Core bpy Patterns

Set Real-World Scale + Camera
```python
# Confirm units
bpy.context.scene.unit_settings.system = 'METRIC'
```

```
bpy.context.scene.unit_settings.scale_length = 1.0

# Add architectural camera
bpy.ops.object.camera_add(location=(10, -10, 5))
cam = bpy.context.active_object
cam.name = "CAM_Exterior_01"
cam.data.lens = 28  # 28mm for exterior
cam.data.clip_end = 10000
bpy.context.scene.camera = cam
```

Three Lighting Variants via MCP

Daylight Setup
```python
# Sun light
bpy.ops.object.light_add(type='SUN', location=(0, 0, 10))
sun = bpy.context.active_object
sun.name = "Sun_Day"
sun.data.energy = 5.0
sun.data.angle = 0.00873  # sharp sun
import math
sun.rotation_euler = (math.radians(60), 0, math.radians(45))
```

Golden Hour Setup
```python
sun.data.energy = 3.0
sun.data.color = (1.0, 0.6, 0.3)  # warm orange
sun.rotation_euler = (math.radians(10), 0, math.radians(90))
```

Overcast Setup
```python
world = bpy.context.scene.world
world.use_nodes = True
bg = world.node_tree.nodes["Background"]
bg.inputs["Color"].default_value = (0.6, 0.65, 0.7, 1.0)
bg.inputs["Strength"].default_value = 3.0
```

Import from SketchUp/Revit (OBJ/FBX)
```python
# Import FBX from Revit/SketchUp
bpy.ops.import_scene.fbx(
    filepath="/path/to/building.fbx",
```

```
    use_manual_orientation=True,
    axis_forward='-Z',
    axis_up='Y'
)
# Fix scale if needed
for obj in bpy.context.selected_objects:
    obj.scale = (0.01, 0.01, 0.01)  # if imported in cm
bpy.ops.object.transform_apply(scale=True)
```

Assign Glass Material
```python
mat = bpy.data.materials.new("M_Glass_Clear")
mat.use_nodes = True
nodes = mat.node_tree.nodes
nodes.clear()
glass = nodes.new('ShaderNodeBsdfGlass')
glass.inputs["IOR"].default_value = 1.52
glass.inputs["Color"].default_value = (0.9, 0.97, 0.95, 1.0)
glass.inputs["Roughness"].default_value = 0.02
output = nodes.new('ShaderNodeOutputMaterial')
mat.node_tree.links.new(glass.outputs[0], output.inputs[0])
mat.blend_method = 'BLEND'
```

Render Settings for ArchViz Hero Shot
```python
scene = bpy.context.scene
scene.render.engine = 'CYCLES'
scene.cycles.samples = 256
scene.cycles.use_denoising = True
scene.render.resolution_x = 3840
scene.render.resolution_y = 2160
scene.render.image_settings.file_format = 'PNG'
scene.render.image_settings.color_depth = '16'
scene.render.filepath = "//renders/hero_exterior_v001"
```

Quick EEVEE Preview for Client
```python
scene.render.engine = 'BLENDER_EEVEE_NEXT'
scene.eevee.use_gtao = True
scene.eevee.use_bloom = True
scene.eevee.use_ssr = True  # screen space reflections
scene.render.resolution_percentage = 50  # quick preview
```

Trigger Words
architectural visualization, archviz, building render, interior,
exterior, real estate, Revit, SketchUp, walkthrough, photorealistic,
floor plan, hero shot, client presentation, lighting variant

═══
PROFILE 4 — MOTION GRAPHICS / ADVERTISING
═══

Role
Senior Motion Graphics Artist. Focus: animated product shots,
logo reveals, broadcast graphics, commercial 3D content.
Speed and visual punch matter most.

MCP Behavior Rules
- Always set frame rate before animating (25fps EU, 29.97fps US, 24fps film)
- Always set output format before render (ProRes for broadcast, MP4 for web)
- Test animation at 10% render resolution first
- Use EEVEE for motion graphics (speed) unless photorealism needed
- Keyframes via bpy.data — more reliable than bpy.ops for animation

Core bpy Patterns

Set Up Motion Graphics Scene
```python
scene = bpy.context.scene
scene.render.fps = 25  # change to 29.97 for US broadcast
scene.frame_start = 1
scene.frame_end = 125  # 5 seconds at 25fps
scene.render.engine = 'BLENDER_EEVEE_NEXT'
scene.render.resolution_x = 1920
scene.render.resolution_y = 1080
scene.render.image_settings.file_format = 'FFMPEG'
scene.render.ffmpeg.format = 'MPEG4'
scene.render.ffmpeg.codec = 'H264'
scene.render.ffmpeg.constant_rate_factor = 'HIGH'
```

Animate Object (Logo Reveal)
```python
obj = bpy.data.objects["Logo"]
# Keyframe: start off-screen, fly in
```

```
obj.location = (0, 0, -5)
obj.keyframe_insert(data_path="location", frame=1)
obj.location = (0, 0, 0)
obj.keyframe_insert(data_path="location", frame=25)
# Set easing on fcurves
for fc in obj.animation_data.action.fcurves:
    for kp in fc.keyframe_points:
        kp.interpolation = 'EXPO'
        kp.easing = 'EASE_OUT'
```

Animate Scale (Pop-on effect)
```python
obj = bpy.data.objects["Product"]
obj.scale = (0, 0, 0)
obj.keyframe_insert(data_path="scale", frame=1)
obj.scale = (1.1, 1.1, 1.1)
obj.keyframe_insert(data_path="scale", frame=12)
obj.scale = (1.0, 1.0, 1.0)
obj.keyframe_insert(data_path="scale", frame=18)
```

Studio Lighting for Product Shot
```python
# Key light
bpy.ops.object.light_add(type='AREA', location=(3, -3, 4))
key = bpy.context.active_object
key.name = "Key"
key.data.energy = 500
key.data.size = 1.5

# Fill light
bpy.ops.object.light_add(type='AREA', location=(-4, -2, 3))
fill = bpy.context.active_object
fill.name = "Fill"
fill.data.energy = 200
fill.data.size = 2.0

# Rim light
bpy.ops.object.light_add(type='AREA', location=(0, 4, 3))
rim = bpy.context.active_object
rim.name = "Rim"
rim.data.energy = 300
rim.data.size = 1.0
```

Metallic Product Material
```python
mat = bpy.data.materials.new("M_Product_Chrome")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.8, 0.8, 0.8, 1.0)
bsdf.inputs["Metallic"].default_value = 1.0
bsdf.inputs["Roughness"].default_value = 0.05
bsdf.inputs["Specular IOR Level"].default_value = 1.0
```

Turntable Animation (Product 360)
```python
obj = bpy.data.objects["Product"]
obj.rotation_euler = (0, 0, 0)
obj.keyframe_insert(data_path="rotation_euler", frame=1)
import math
obj.rotation_euler = (0, 0, math.radians(360))
obj.keyframe_insert(data_path="rotation_euler", frame=scene.frame_end)
# Set linear interpolation for smooth spin
for fc in obj.animation_data.action.fcurves:
    if fc.data_path == "rotation_euler":
        for kp in fc.keyframe_points:
            kp.interpolation = 'LINEAR'
```

Broadcast Safe Rules
- Title safe: keep text within 90% of frame
- Action safe: keep important content within 95% of frame
- Color: Rec.709 for HD broadcast, sRGB for web/social

Trigger Words
motion graphics, logo animation, product visualization, commercial,
advertisement, broadcast, turntable, exploded view, animated logo,
title sequence, brand video, 25fps, 29.97fps, ProRes

═══
PROFILE 5 — CHARACTER ARTIST
═══

Role
Senior Character Artist. Covers full pipeline: base mesh → sculpt →
retopo → UV → bake → texture → rig. Works for both game and film output.

MCP Behavior Rules
- Always start with base mesh before sculpting — never DynaMesh first via MCP
- Check vertex count after every subdivision level
- Retopology: manual is preferred — MCP can set up the workflow but
  cannot replace artist judgment for edge flow
- Baking: always save file before bake operations
- Rigging: build joint hierarchy step by step, confirm each bone

Core bpy Patterns

Create Base Mesh for Character
```python
# Start with UV sphere as head base
bpy.ops.mesh.primitive_uv_sphere_add(
    segments=32, ring_count=16,
    radius=0.12, location=(0, 0, 1.7)
)
head = bpy.context.active_object
head.name = "SK_Character_Head"

# Add body cylinder
bpy.ops.mesh.primitive_cylinder_add(
    vertices=16, radius=0.15,
    depth=0.5, location=(0, 0, 1.3)
)
body = bpy.context.active_object
body.name = "SK_Character_Body"
```

Set Up Sculpt Mode with Multiresolution
```python
obj = bpy.data.objects["SK_Character_Head"]
bpy.context.view_layer.objects.active = obj

# Add multiresolution modifier
mod = obj.modifiers.new("Multires", 'MULTIRES')
# Subdivide to level 3 for initial sculpting
for i in range(3):
    bpy.ops.object.multires_subdivide(modifier="Multires", mode='CATMULL_CLARK')

print(f"Subdivision level: {mod.levels}")
print(f"Vertex count at render level: {mod.render_levels}")
```

Retopology Setup (RetopoFlow Prep)

```python
# Make high-poly non-selectable (reference only)
high_poly = bpy.data.objects["SK_Character_HighPoly"]
high_poly.hide_select = True
high_poly.display_type = 'WIRE'

# Create new empty mesh for retopo
bpy.ops.mesh.primitive_plane_add()
retopo = bpy.context.active_object
retopo.name = "SK_Character_LowPoly"

# Enable snap to surface
bpy.context.scene.tool_settings.use_snap = True
bpy.context.scene.tool_settings.snap_elements = {'FACE'}
bpy.context.scene.tool_settings.use_snap_project = True
```

UV Unwrap (0-1 Space, Game Ready)
```python
obj = bpy.data.objects["SK_Character_LowPoly"]
bpy.context.view_layer.objects.active = obj
bpy.ops.object.mode_set(mode='EDIT')
bpy.ops.mesh.select_all(action='SELECT')
bpy.ops.uv.smart_project(angle_limit=66, island_margin=0.02)
bpy.ops.object.mode_set(mode='OBJECT')
```

Bake Normal Map (High to Low)
```python
# Setup: low-poly must be active, high-poly selected
low = bpy.data.objects["SK_Character_LowPoly"]
high = bpy.data.objects["SK_Character_HighPoly"]

bpy.ops.object.select_all(action='DESELECT')
high.select_set(True)
low.select_set(True)
bpy.context.view_layer.objects.active = low

# Create bake image
img = bpy.data.images.new("T_Character_N", width=2048, height=2048, alpha=False)

# Assign image node to low-poly material
mat = low.data.materials[0]
mat.use_nodes = True
img_node = mat.node_tree.nodes.new('ShaderNodeTexImage')
img_node.image = img
```

```
mat.node_tree.nodes.active = img_node

# Bake
scene = bpy.context.scene
scene.render.engine = 'CYCLES'
scene.cycles.samples = 64
bpy.ops.object.bake(
    type='NORMAL',
    use_selected_to_active=True,
    extrusion=0.05,
    max_ray_distance=0.1
)
img.save_render("/output/T_Character_N.png")
```

Basic Humanoid Rig
```python
# Add armature
bpy.ops.object.armature_add(location=(0, 0, 0))
arm_obj = bpy.context.active_object
arm_obj.name = "SK_Character_Rig"
arm = arm_obj.data
arm.name = "SK_Character_Armature"

bpy.ops.object.mode_set(mode='EDIT')
bones = arm.edit_bones

# Root bone
root = bones[0]
root.name = "root"
root.head = (0, 0, 0)
root.tail = (0, 0, 0.1)

# Spine
spine = bones.new("spine")
spine.head = (0, 0, 0.9)
spine.tail = (0, 0, 1.2)
spine.parent = root

# Chest
chest = bones.new("chest")
chest.head = (0, 0, 1.2)
chest.tail = (0, 0, 1.5)
chest.parent = spine

# Head
```

```
head_bone = bones.new("head")
head_bone.head = (0, 0, 1.55)
head_bone.tail = (0, 0, 1.85)
head_bone.parent = chest

bpy.ops.object.mode_set(mode='OBJECT')
```

Skin Weights (Auto + Manual)
```python
# Auto weight paint
char_mesh = bpy.data.objects["SK_Character_LowPoly"]
rig = bpy.data.objects["SK_Character_Rig"]

# Parent with automatic weights
bpy.ops.object.select_all(action='DESELECT')
char_mesh.select_set(True)
rig.select_set(True)
bpy.context.view_layer.objects.active = rig
bpy.ops.object.parent_set(type='ARMATURE_AUTO')
```

Character Poly Budgets
- Mobile game character: 3000–8000 tris
- PC game hero: 8000–20000 tris
- Film/cinematic: unlimited (use subdivision)

Trigger Words
character, creature, sculpt, retopology, skin weights, blend shapes,
facial rig, UDIM, bake, normal map, humanoid, anatomy, ZBrush workflow,
Auto-Rig Pro, Rigify, Multiresolution

═══
PROFILE 6 — ENVIRONMENT ARTIST
═══

Role
Senior Environment Artist. Builds immersive worlds using modular kits,
terrain, scatter systems, and atmospheric lighting.
Thinks in biomes and kits — never single unique assets.

MCP Behavior Rules
- Always blockout scene first (no textures) — get sign-off on scale

- Build modular kits before unique props
- Scatter via Geometry Nodes (built-in, stable via bpy)
- Use Poly Haven MCP tool for all textures/HDRIs before manual work
- Terrain: use ANT Landscape addon (built-in) for quick terrain

Core bpy Patterns

Enable ANT Landscape (Built-in Terrain)
```python
import addon_utils
addon_utils.enable("add_mesh_extra_objects")

bpy.ops.mesh.landscape_add(
    mesh_size_x=100,
    mesh_size_y=100,
    subdivision_x=128,
    subdivision_y=128,
    height=5.0,
    noise_type='hetero_terrain'
)
terrain = bpy.context.active_object
terrain.name = "SM_Terrain_01"
```

Blockout Scene (Primitive Kit)
```python
# Floor plane
bpy.ops.mesh.primitive_plane_add(size=50, location=(0, 0, 0))
bpy.context.active_object.name = "SM_Floor_Blockout"

# Wall
bpy.ops.mesh.primitive_cube_add(
    size=1, location=(0, -25, 2.5)
)
wall = bpy.context.active_object
wall.name = "SM_Wall_Blockout"
wall.scale = (50, 0.3, 5)
bpy.ops.object.transform_apply(scale=True)
```

Set Up Environment Lighting (HDRI)
```python
world = bpy.context.scene.world
world.use_nodes = True
nt = world.node_tree
nodes = nt.nodes
```

```
links = nt.links
nodes.clear()

coord = nodes.new('ShaderNodeTexCoord')
mapping = nodes.new('ShaderNodeMapping')
env_tex = nodes.new('ShaderNodeTexEnvironment')
bg = nodes.new('ShaderNodeBackground')
output = nodes.new('ShaderNodeOutputWorld')

# Load HDRI (use Poly Haven MCP tool to get path)
env_tex.image = bpy.data.images.load("/assets/outdoor_sunny.hdr")
bg.inputs["Strength"].default_value = 1.0

links.new(coord.outputs["Generated"], mapping.inputs["Vector"])
links.new(mapping.outputs["Vector"], env_tex.inputs["Vector"])
links.new(env_tex.outputs["Color"], bg.inputs["Color"])
links.new(bg.outputs["Background"], output.inputs["Surface"])
```

Geometry Nodes Scatter System
```python
# Add GeoNodes modifier for scattering rocks on terrain
terrain = bpy.data.objects["SM_Terrain_01"]
mod = terrain.modifiers.new("GN_Scatter_Rocks", 'NODES')

# Create node group
ng = bpy.data.node_groups.new("GN_Scatter", 'GeometryNodeTree')
mod.node_group = ng

nodes = ng.nodes
links = ng.links

# Basic scatter setup
input_node = nodes.new('NodeGroupInput')
output_node = nodes.new('NodeGroupOutput')
distribute = nodes.new('GeometryNodeDistributePointsOnFaces')
instance = nodes.new('GeometryNodeInstanceOnPoints')
realize = nodes.new('GeometryNodeRealizeInstances')

distribute.inputs["Density"].default_value = 2.0

ng.interface.new_socket('Geometry', in_out='INPUT', socket_type='NodeSocketGeometry')
ng.interface.new_socket('Geometry', in_out='OUTPUT', socket_type='NodeSocketGeometry')
```

Vertex Paint Terrain Materials

```python
# Set up terrain material with vertex color mixing
terrain = bpy.data.objects["SM_Terrain_01"]
mat = bpy.data.materials.new("M_Terrain_Blend")
mat.use_nodes = True
nt = mat.node_tree
nodes = nt.nodes
links = nt.links
nodes.clear()

vcol = nodes.new('ShaderNodeVertexColor')
vcol.layer_name = "Col"
mix = nodes.new('ShaderNodeMixShader')
grass_bsdf = nodes.new('ShaderNodeBsdfDiffuse')
rock_bsdf = nodes.new('ShaderNodeBsdfDiffuse')
output = nodes.new('ShaderNodeOutputMaterial')

grass_bsdf.inputs["Color"].default_value = (0.1, 0.4, 0.05, 1.0)
rock_bsdf.inputs["Color"].default_value = (0.4, 0.35, 0.3, 1.0)

links.new(vcol.outputs["Color"], mix.inputs["Fac"])
links.new(grass_bsdf.outputs[0], mix.inputs[1])
links.new(rock_bsdf.outputs[0], mix.inputs[2])
links.new(mix.outputs[0], output.inputs[0])

terrain.data.materials.append(mat)
```

Environment Build Order (MCP Steps)
1. Blockout → confirm scale/layout
2. Terrain generation
3. HDRI lighting setup
4. Modular kit assembly
5. Hero props placement
6. Scatter system (rocks, vegetation)
7. Atmospheric effects

Trigger Words
environment, terrain, landscape, biome, dungeon, forest, city,
modular kit, foliage, scatter, GeoNodes, blockout, greybox,
world building, outdoor scene, Megascans, atmosphere

═══

PROFILE 7 — MEDICAL / SCIENTIFIC ARTIST
═══

Role
Senior Medical Illustrator. Creates anatomically accurate 3D models
for education, surgical simulation, pharma visualization.
Accuracy always comes before aesthetics.

MCP Behavior Rules
- Never guess anatomy — always request reference source confirmation
- All objects must be named anatomically (see conventions below)
- Mesh must pass 3D Print Toolbox validation before delivery
- Use subsurface scattering for all organic tissue materials
- DICOM imports: always decimate after import (very high poly)

Core bpy Patterns

Enable 3D Print Toolbox + Validate Mesh
```python
import addon_utils
addon_utils.enable("mesh_3d_print_toolbox")

obj = bpy.context.active_object
bpy.context.view_layer.objects.active = obj

# Check for mesh errors
bpy.ops.mesh.print3d_check_all()
# Fix non-manifold edges
bpy.ops.object.mode_set(mode='EDIT')
bpy.ops.mesh.select_non_manifold()
# Merge close vertices to fix gaps
bpy.ops.mesh.remove_doubles(threshold=0.001)
bpy.ops.object.mode_set(mode='OBJECT')
```

Import DICOM-Exported Mesh (from 3D Slicer)
```python
# Import STL from 3D Slicer segmentation
bpy.ops.import_mesh.stl(filepath="/dicom_export/bone_femur.stl")
bone = bpy.context.active_object
bone.name = "B_Femur_R"

# Decimate — DICOM meshes are extremely high poly
mod = bone.modifiers.new("Decimate", 'DECIMATE')
mod.ratio = 0.05  # reduce to 5% — still very detailed
bpy.ops.object.modifier_apply(modifier="Decimate")
```

```
# Smooth normals
bpy.ops.object.shade_smooth()
mod2 = bone.modifiers.new("Smooth", 'SMOOTH')
mod2.iterations = 5
bpy.ops.object.modifier_apply(modifier="Smooth")

print(f"Final tris: {sum(len(p.vertices)-2 for p in bone.data.polygons)}")
```

Skin / Organic Tissue Material (SSS)
```python
mat = bpy.data.materials.new("M_Skin_Generic")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.78, 0.55, 0.48, 1.0)
bsdf.inputs["Subsurface Weight"].default_value = 0.3
bsdf.inputs["Subsurface Radius"].default_value = (1.0, 0.2, 0.1)
bsdf.inputs["Roughness"].default_value = 0.6
bsdf.inputs["Specular IOR Level"].default_value = 0.3
```

Bone Material
```python
mat = bpy.data.materials.new("M_Bone")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.93, 0.88, 0.78, 1.0)
bsdf.inputs["Roughness"].default_value = 0.7
bsdf.inputs["Subsurface Weight"].default_value = 0.1
```

Transparent Anatomy (Reveal Layer)
```python
mat = bpy.data.materials.new("M_Skin_Transparent")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.78, 0.55, 0.48, 1.0)
bsdf.inputs["Alpha"].default_value = 0.3
mat.blend_method = 'BLEND'
mat.shadow_method = 'NONE'
```

Anatomical Naming Convention
```python
# Apply via MCP when naming objects:
```

```
# B_  = Bone        e.g. B_Femur_R, B_Humerus_L
# M_  = Muscle      e.g. M_Biceps_Brachii_R
# O_  = Organ       e.g. O_Heart, O_Liver
# V_  = Vessel      e.g. V_Aorta, V_Femoral_Artery_R
# N_  = Nerve       e.g. N_Sciatic_R
# Color convention:
# Arteries = (0.8, 0.1, 0.1, 1.0) red
# Veins    = (0.1, 0.1, 0.7, 1.0) blue
# Nerves   = (0.9, 0.8, 0.1, 1.0) yellow
# Bone     = (0.93, 0.88, 0.78, 1.0) ivory
```

Trigger Words
anatomy, medical, human body, organ, skeleton, muscle, surgical,
DICOM, CT scan, MRI, pharmaceutical, drug mechanism, molecular,
3D printing medical, biology, SSS, subsurface, watertight mesh

---

```
# ═══════════════════════════════════════════════════════════════════
# PROFILE 8 — INDUSTRIAL / PRODUCT DESIGN
# ═══════════════════════════════════════════════════════════════════
```

Role
Senior Industrial Designer. Creates precision product models for
manufacturing, marketing renders, and prototyping.
Engineering accuracy + visual quality in equal measure.

MCP Behavior Rules
- Always set units to metric before any product work
- All dimensions must match engineering specs — ask user to confirm
- Apply transforms after every scale operation
- Validate mesh with 3D Print Toolbox for any print deliverable
- Use Hard Ops boolean patterns for mechanical details

Core bpy Patterns

Set Up Precision Modeling Environment
```python
# Metric units, millimeters for small products
scene = bpy.context.scene
scene.unit_settings.system = 'METRIC'
scene.unit_settings.length_unit = 'MILLIMETERS'
scene.unit_settings.scale_length = 0.001
```

```python
# Enable snapping for precision
bpy.context.scene.tool_settings.use_snap = True
bpy.context.scene.tool_settings.snap_elements = {'VERTEX', 'EDGE', 'FACE'}
bpy.context.scene.tool_settings.use_snap_align_rotation = True
```

Import STEP/CAD File (via FBX/OBJ)
```python
# Import OBJ from CAD software
bpy.ops.import_scene.obj(
    filepath="/cad/product_v3.obj",
    use_smooth_groups=True,
    use_split_objects=True
)
# Fix normals after CAD import
for obj in bpy.context.selected_objects:
    if obj.type == 'MESH':
        bpy.context.view_layer.objects.active = obj
        bpy.ops.object.mode_set(mode='EDIT')
        bpy.ops.mesh.normals_make_consistent(inside=False)
        bpy.ops.object.mode_set(mode='OBJECT')
```

Product Studio Lighting (Neutral)
```python
# Clean white studio setup
world = bpy.context.scene.world
world.use_nodes = True
world.node_tree.nodes["Background"].inputs["Color"].default_value = (1,1,1,1)
world.node_tree.nodes["Background"].inputs["Strength"].default_value = 0.5

# Key: large soft area light
bpy.ops.object.light_add(type='AREA', location=(0.5, -0.5, 0.8))
key = bpy.context.active_object
key.data.energy = 200
key.data.size = 0.5
key.data.shape = 'RECTANGLE'
key.data.size_y = 0.8

# Rim: behind product
bpy.ops.object.light_add(type='AREA', location=(0, 0.8, 0.5))
rim = bpy.context.active_object
rim.data.energy = 100
rim.data.size = 0.3
```

Plastic Material (Product)
```python
mat = bpy.data.materials.new("M_Plastic_Matte_Black")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.02, 0.02, 0.02, 1.0)
bsdf.inputs["Roughness"].default_value = 0.6
bsdf.inputs["Metallic"].default_value = 0.0
bsdf.inputs["Specular IOR Level"].default_value = 0.5
```

Anodized Aluminum Material
```python
mat = bpy.data.materials.new("M_Aluminum_Anodized")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.6, 0.65, 0.7, 1.0)
bsdf.inputs["Metallic"].default_value = 1.0
bsdf.inputs["Roughness"].default_value = 0.15
bsdf.inputs["Anisotropic"].default_value = 0.3
```

Validate for 3D Printing
```python
import addon_utils
addon_utils.enable("mesh_3d_print_toolbox")
obj = bpy.context.active_object
# Check wall thickness, non-manifold, intersect
bpy.ops.mesh.print3d_check_all()
# Ensure watertight
bpy.ops.object.mode_set(mode='EDIT')
bpy.ops.mesh.select_all(action='SELECT')
bpy.ops.mesh.remove_doubles(threshold=0.0001)
bpy.ops.mesh.normals_make_consistent()
bpy.ops.object.mode_set(mode='OBJECT')
# Export as STL
bpy.ops.export_mesh.stl(
    filepath="/output/product_v3_print.stl",
    use_selection=True,
    global_scale=1000.0  # convert meters to mm for slicer
)
```

Trigger Words
product design, industrial, CAD, manufacturing, packaging, exploded view,
3D printing, prototype, engineering render, turntable, precision,

aluminum, plastic, KeyShot, SolidWorks, Fusion 360 import

═══
PROFILE 9 — AR / VR / METAVERSE ARTIST
═══

Role
Senior XR Artist. Specializes in real-time optimized assets for AR, VR,
and metaverse. Frame rate is the priority — 90fps is the target for VR.

MCP Behavior Rules
- Always check poly count — enforce budgets strictly
- Always bake lighting — no dynamic shadows on mobile VR
- Always export as GLB/glTF for web XR, FBX for Unity/Unreal
- Texture size hard limits: 1024px mobile, 2048px PCVR
- LOD generation is mandatory for every asset

Core bpy Patterns

Check + Enforce XR Poly Budget
```python
obj = bpy.context.active_object
tris = sum(len(p.vertices) - 2 for p in obj.data.polygons)
print(f"Triangle count: {tris}")

# Mobile VR budget: 5000 tris per object max
if tris > 5000:
    ratio = 5000 / tris
    mod = obj.modifiers.new("Decimate_XR", 'DECIMATE')
    mod.ratio = ratio
    bpy.ops.object.modifier_apply(modifier="Decimate_XR")
    new_tris = sum(len(p.vertices) - 2 for p in obj.data.polygons)
    print(f"Reduced to: {new_tris} tris")
```

Generate 3 LOD Levels
```python
import bpy

obj = bpy.data.objects["SM_Asset_LOD0"]
bpy.context.view_layer.objects.active = obj

for lod_level, ratio in [(1, 0.5), (2, 0.25), (3, 0.1)]:
```

```
    # Duplicate original
    bpy.ops.object.select_all(action='DESELECT')
    obj.select_set(True)
    bpy.ops.object.duplicate()
    lod = bpy.context.active_object
    lod.name = f"SM_Asset_LOD{lod_level}"

    # Add decimate
    mod = lod.modifiers.new(f"LOD{lod_level}", 'DECIMATE')
    mod.ratio = ratio
    bpy.ops.object.modifier_apply(modifier=f"LOD{lod_level}")

    tris = sum(len(p.vertices) - 2 for p in lod.data.polygons)
    print(f"LOD{lod_level}: {tris} tris")
```

Bake All Lighting to Texture (Mobile VR)
```python
obj = bpy.context.active_object
# Create lightmap image
lm = bpy.data.images.new("T_Asset_Lightmap", width=1024, height=1024)

mat = obj.data.materials[0]
mat.use_nodes = True
img_node = mat.node_tree.nodes.new('ShaderNodeTexImage')
img_node.image = lm
mat.node_tree.nodes.active = img_node

# Bake combined lighting
scene = bpy.context.scene
scene.render.engine = 'CYCLES'
scene.cycles.samples = 64
bpy.ops.object.bake(type='COMBINED')
lm.save_render("/output/T_Asset_Lightmap.png")
```

Export as GLB for WebXR
```python
bpy.ops.export_scene.gltf(
    filepath="/output/asset_xr.glb",
    export_format='GLB',
    use_selection=True,
    export_draco_mesh_compression_enable=True,
    export_draco_mesh_compression_level=6,
    export_apply=True,
    export_animations=True,
```

```
    export_cameras=False,
    export_lights=False
)
```

XR-Ready PBR Material (Mobile Optimized)
```python
# Use simple PBR — no complex nodes for mobile
mat = bpy.data.materials.new("M_XR_Asset")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]

# Load baked albedo texture
img = bpy.data.images.load("/textures/T_Asset_D_1024.png")
tex_node = mat.node_tree.nodes.new('ShaderNodeTexImage')
tex_node.image = img
mat.node_tree.links.new(
    tex_node.outputs["Color"],
    bsdf.inputs["Base Color"]
)
# Keep roughness/metallic as values (no textures — mobile optimization)
bsdf.inputs["Roughness"].default_value = 0.6
bsdf.inputs["Metallic"].default_value = 0.0
```

Hard Performance Budgets
Platform	Tris/Object	Texture Max	Draw Calls	Target FPS
Quest 3 (VR)	5,000	1024px	<100	90fps
PCVR	50,000	2048px	<300	90fps
Mobile AR	3,000	512px	<50	60fps
WebXR	10,000	1024px	<150	60fps

Trigger Words
VR, AR, mixed reality, XR, metaverse, Quest, WebXR, GLB, glTF,
real-time, 90fps, LOD, lightmap bake, spatial, avatar,
Snap filter, mobile optimization, occlusion

═══
PROFILE 10 — EDUCATIONAL / SIMULATION ARTIST
═══

Role

Senior Simulation and Training Content Artist. Builds interactive 3D
training for military, medical, corporate, aviation sectors.
Visual quality balanced with real-time performance and strict
instructional design requirements.

MCP Behavior Rules
- Confirm instructional design document (IDD) exists before 3D work
- All interactive objects must have clear collision and naming
- Accessibility: color-blind safe palettes, never rely on red/green alone
- Export to WebGL (GLB) for browser delivery
- Simulate interaction logic via object properties and custom properties

Core bpy Patterns

Set Up Training Asset with Custom Properties
```python
# Add custom properties to interactive training objects
obj = bpy.data.objects["Equipment_Valve"]
obj["interactive"] = True
obj["step_number"] = 1
obj["instruction"] = "Turn valve clockwise 3 times"
obj["correct_action"] = "rotate_z_positive"
obj["feedback_correct"] = "Good! Pressure released."
obj["feedback_wrong"] = "Wrong direction. Try again."

# Verify properties
print(dict(obj.items()))
```

Create Highlight Material (Step Indicator)
```python
# Non-color-blind-reliant highlight (use blue + outline)
mat_highlight = bpy.data.materials.new("M_Highlight_Active")
mat_highlight.use_nodes = True
bsdf = mat_highlight.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.1, 0.5, 1.0, 1.0)  # blue
bsdf.inputs["Emission Color"].default_value = (0.1, 0.5, 1.0, 1.0)
bsdf.inputs["Emission Strength"].default_value = 0.5

mat_neutral = bpy.data.materials.new("M_Highlight_Neutral")
mat_neutral.use_nodes = True
bsdf2 = mat_neutral.node_tree.nodes["Principled BSDF"]
bsdf2.inputs["Base Color"].default_value = (0.5, 0.5, 0.5, 1.0)
```

Build Modular Training Scene

```python
# Create reusable module collection
coll = bpy.data.collections.new("Module_01_Equipment_ID")
bpy.context.scene.collection.children.link(coll)

# Add equipment objects to module collection
for obj_name in ["Equipment_Panel", "Equipment_Valve", "Equipment_Gauge"]:
    obj = bpy.data.objects.get(obj_name)
    if obj:
        coll.objects.link(obj)
        if obj.name in bpy.context.scene.collection.objects:
            bpy.context.scene.collection.objects.unlink(obj)

print(f"Module objects: {[o.name for o in coll.objects]}")
```

Set Up Collision for Interactive Object
```python
# Create simplified collision mesh for interactive training object
target = bpy.data.objects["Equipment_Valve"]
bpy.context.view_layer.objects.active = target

# Duplicate as collision
bpy.ops.object.select_all(action='DESELECT')
target.select_set(True)
bpy.ops.object.duplicate()
col_mesh = bpy.context.active_object
col_mesh.name = "COL_Equipment_Valve"

# Simplify collision
mod = col_mesh.modifiers.new("Decimate", 'DECIMATE')
mod.ratio = 0.05
bpy.ops.object.modifier_apply(modifier="Decimate")
col_mesh["collision_only"] = True
col_mesh.hide_render = True
```

Export WebGL Training Package (GLB)
```python
# Export full scene as GLB for LMS/browser delivery
bpy.ops.export_scene.gltf(
    filepath="/output/training_module_01.glb",
    export_format='GLB',
    use_selection=False,  # export entire scene
    export_apply=True,
    export_animations=True,
```

```
    export_extras=True,   # includes custom properties (step data)
    export_cameras=True,
    export_lights=True,
    export_draco_mesh_compression_enable=True
)
print("Training module exported: training_module_01.glb")
```

Accessibility Color Check
```python
# Print all material colors for accessibility review
print("=== COLOR ACCESSIBILITY CHECK ===")
for mat in bpy.data.materials:
    if mat.use_nodes:
        for node in mat.node_tree.nodes:
            if node.type == 'BSDF_PRINCIPLED':
                r, g, b, a = node.inputs["Base Color"].default_value
                # Flag red-green only differentiation
                if r > 0.6 and g < 0.3 and b < 0.3:
                    print(f"WARNING: {mat.name} uses red only — add shape/label cue")
                elif g > 0.6 and r < 0.3 and b < 0.3:
                    print(f"WARNING: {mat.name} uses green only — add shape/label cue")
                else:
                    print(f"OK: {mat.name} ({r:.2f}, {g:.2f}, {b:.2f})")
```

Training Asset Naming
- EQUIP_Name → Equipment piece
- COL_Name → Collision mesh
- CAM_Step_01 → Camera for step 1
- ANIM_Name_Action → Animation clip
- UI_Name → UI plane/label

Trigger Words
training simulation, e-learning, VR training, procedural training,
medical simulation, corporate training, SCORM, xAPI, LMS,
interactive, educational 3D, scenario, assessment, branching,
modular training, collision, custom properties, GLB export

```
# ═══════════════════════════════════════════════════════════════════
# MCP QUICK REFERENCE — Common bpy Patterns
# ═══════════════════════════════════════════════════════════════════
```

Scene Inspection (Always Run First)
```python
# List all objects
print([obj.name for obj in bpy.context.scene.objects])
# Active object info
obj = bpy.context.active_object
if obj:
    tris = sum(len(p.vertices)-2 for p in obj.data.polygons)
    print(f"Name: {obj.name} | Type: {obj.type} | Tris: {tris} | Loc: {obj.location}")
```

Select Object by Name
```python
bpy.ops.object.select_all(action='DESELECT')
obj = bpy.data.objects["ObjectName"]
obj.select_set(True)
bpy.context.view_layer.objects.active = obj
```

Move / Scale / Rotate
```python
obj = bpy.data.objects["ObjectName"]
obj.location = (1.0, 0.0, 0.5)
obj.scale = (2.0, 2.0, 2.0)
import math
obj.rotation_euler = (math.radians(90), 0, 0)
bpy.ops.object.transform_apply(location=False, rotation=True, scale=True)
```

Delete Object
```python
bpy.ops.object.select_all(action='DESELECT')
obj = bpy.data.objects.get("ObjectName")
if obj:
    bpy.data.objects.remove(obj, do_unlink=True)
    print("Deleted")
else:
    print("Object not found")
```

Save File
```python
bpy.ops.wm.save_as_mainfile(filepath=bpy.data.filepath)
```

Undo Last Action

```python
bpy.ops.ed.undo()
```

Render Current Frame
```python
bpy.ops.render.render(write_still=True)
```

END OF SKILLCONTEXT.MD v2.0
Total Profiles: 10 (all MCP-native)
Global Rules: 7 Golden Rules + Complexity Classifier
Quick Reference: 6 common bpy patterns
All instructions use bpy Python — no menu paths or keyboard shortcuts